

实验三 基于Verilog HDL的数字电路设计（三）

实验报告

1 实验原理

1.1 简易数字钟系统

简易数字钟系统是一种典型的综合性时序逻辑与同步控制系统。它通常由时钟分频模块、计时核心模块、显示控制与驱动模块等部分构成。系统以开发板上的高频晶振作为参考时钟，通过分频电路产生标准的 1Hz 脉冲用于精确计时，同时也产生适当频率的扫描时钟用于驱动外部显示设备。

计时核心模块通过级联的BCD码计数器记录时、分、秒等信息。当最低位的秒计数器达到进位条件（通常为60进1）时，向分钟计数器输出进位脉冲，以此类推。在支持调整和闹钟功能的系统中，还会引入多路选择器和组合逻辑，根据当前的工作模式（*Mode*）将对应的时间数据或告警设定数据送入显示通路。

本实验中，数字钟的显示输出创新性地采用了VGA接口。VGA时序控制模块根据预设的显示分辨率（如 $640 \times 480@60\text{Hz}$ ）产生精确的行同步（*Hsync*）与场同步（*Vsync*）信号，并协同动画生成模块或字库模块将当前的时间数据映射为屏幕特定坐标范围内的像素颜色（RGB）数据，从而实现时间的直观可视化。

2 主要程序代码

clock_core.v:

```
1 module clock_core (
2     input wire      clk,
3     input wire      rst,
4     input wire      btn_hour_up,
5     input wire      btn_hour_down,
6     input wire      btn_min_up,
7     input wire      btn_min_down,
8     input wire      sw_mode,
9     input wire      alarm_en,
10    output reg [4:0] hour,
11    output reg [5:0] min,
12    output reg [5:0] sec,
13    output wire [4:0] disp_hour,
14    output wire [5:0] disp_min,
15    output wire [5:0] disp_sec,
16    output reg      led_hourly,
17    output reg      led_alarm,
18    output reg      one_hz_tick
19 );
20
21    localparam CLK_HZ = 25_174_010;
22
23    reg [24:0] tick_cnt;
```

```

24 reg [4:0] alarm_hour;
25 reg [5:0] alarm_min;
26 reg [1:0] mode_sync;
27 reg [1:0] alarm_en_sync;
28
29 wire hour_up_pulse;
30 wire hour_down_pulse;
31 wire min_up_pulse;
32 wire min_down_pulse;
33 wire mode;
34 wire alarm_enabled;
35
36 // 两级同步处理异步输入信号
37 assign mode = mode_sync[1];
38 assign alarm_enabled = alarm_en_sync[1];
39
40 // 根据当前模式选择显示的时间：为闹钟，为正常时钟10
41 assign disp_hour = mode ? alarm_hour : hour;
42 assign disp_min = mode ? alarm_min : min;
43 assign disp_sec = mode ? 6'd0 : sec;
44
45 // 实例化按键消抖及脉冲生成模块
46 button_pulse u_btn_hour_up (
47     .clk (clk),
48     .rst (rst),
49     .button_in (btn_hour_up),
50     .pulse (hour_up_pulse)
51 );
52
53 button_pulse u_btn_hour_down (
54     .clk (clk),
55     .rst (rst),
56     .button_in (btn_hour_down),
57     .pulse (hour_down_pulse)
58 );
59
60 button_pulse u_btn_min_up (
61     .clk (clk),
62     .rst (rst),
63     .button_in (btn_min_up),
64     .pulse (min_up_pulse)
65 );
66
67 button_pulse u_btn_min_down (
68     .clk (clk),
69     .rst (rst),
70     .button_in (btn_min_down),
71     .pulse (min_down_pulse)
72 );
73
74 always @(posedge clk) begin
75     if (rst) begin
76         tick_cnt <= 25'd0;
77         hour <= 5'd0;
78         min <= 6'd0;
79         sec <= 6'd0;
80         alarm_hour <= 5'd0;
81         alarm_min <= 6'd0;

```

```

82         mode_sync    <= 2'b00;
83         alarm_en_sync <= 2'b00;
84         led_hourly    <= 1'b0;
85         led_alarm     <= 1'b0;
86         one_hz_tick   <= 1'b0;
87     end else begin
88         one_hz_tick <= 1'b0;
89         // 开关信号的同步处理
90         mode_sync <= {mode_sync[0], sw_mode};
91         alarm_en_sync <= {alarm_en_sync[0], alarm_en};
92
93         // 1Hz 脉冲产生及自然计时进位逻辑
94         if (tick_cnt == CLK_HZ - 1) begin
95             tick_cnt    <= 25'd0;
96             one_hz_tick <= 1'b1;
97
98             if (sec == 6'd59) begin
99                 sec <= 6'd0;
100                 if (min == 6'd59) begin
101                     min <= 6'd0;
102                     if (hour == 5'd23) begin
103                         hour <= 5'd0;
104                     end else begin
105                         hour <= hour + 5'd1;
106                     end
107                 end else begin
108                     min <= min + 6'd1;
109                 end
110             end else begin
111                 sec <= sec + 6'd1;
112             end
113         end else begin
114             tick_cnt <= tick_cnt + 25'd1;
115         end
116
117         // 按键手动时间调整逻辑
118         if (mode) begin
119             // 闹钟模式下的时间设定
120             if (hour_up_pulse) begin
121                 alarm_hour <= (alarm_hour == 5'd23) ? 5'd0 :
122                 alarm_hour + 5'd1;
123             end else if (hour_down_pulse) begin
124                 alarm_hour <= (alarm_hour == 5'd0) ? 5'd23 :
125                 alarm_hour - 5'd1;
126             end
127
128             if (min_up_pulse) begin
129                 alarm_min <= (alarm_min == 6'd59) ? 6'd0 : alarm_min
130                 + 6'd1;
131             end else if (min_down_pulse) begin
132                 alarm_min <= (alarm_min == 6'd0) ? 6'd59 : alarm_min
133                 - 6'd1;
134             end
135         end else begin
136             // 正常时钟模式下的时间设定
137             if (hour_up_pulse) begin
138                 hour <= (hour == 5'd23) ? 5'd0 : hour + 5'd1;
139             end
140             if (min_up_pulse) begin
141                 min <= (min == 6'd59) ? 6'd0 : min + 6'd1;
142             end
143             if (min_down_pulse) begin
144                 min <= (min == 6'd0) ? 6'd59 : min - 6'd1;
145             end
146             if (sec_up_pulse) begin
147                 sec <= (sec == 6'd59) ? 6'd0 : sec + 6'd1;
148             end
149             if (sec_down_pulse) begin
150                 sec <= (sec == 6'd0) ? 6'd59 : sec - 6'd1;
151             end
152         end
153     end
154 end

```

```

136         end else if (hour_down_pulse) begin
137             hour <= (hour == 5'd0) ? 5'd23 : hour - 5'd1;
138             sec <= 6'd0;
139         end
140
141         if (min_up_pulse) begin
142             min <= (min == 6'd59) ? 6'd0 : min + 6'd1;
143             sec <= 6'd0;
144         end else if (min_down_pulse) begin
145             min <= (min == 6'd0) ? 6'd59 : min - 6'd1;
146             sec <= 6'd0;
147         end
148     end
149
150     // 指示灯逻辑: 整点报时以及闹钟触发
151     led_hourly <= (min == 6'd0) && (sec == 6'd0);
152     led_alarm <= alarm_enabled && (hour == alarm_hour) && (min
== alarm_min);
153     end
154 end
155 endmodule

```

Listing 1: clock_core.v

anim_gen.v:

```

1 module anim_gen (
2     input wire      clk,
3     input wire      video_on,
4     input wire [9:0] pixel_x,
5     input wire [9:0] pixel_y,
6     input wire      sw_mode,
7     input wire      alarm_en,
8     input wire      alarm_active,
9     input wire [4:0] hour,
10    input wire [5:0] min,
11    input wire [5:0] sec,
12    input wire [3:0] bg_r,
13    input wire [3:0] bg_g,
14    input wire [3:0] bg_b,
15    output reg [3:0] vga_r,
16    output reg [3:0] vga_g,
17    output reg [3:0] vga_b
18 );
19 wire text_on;
20 wire text_shadow_on;
21 wire colon_on;
22 wire colon_shadow_on;
23 wire [3:0] hour_tens;
24 wire [3:0] hour_ones;
25 wire [3:0] min_tens;
26 wire [3:0] min_ones;
27 wire [3:0] sec_tens;
28 wire [3:0] sec_ones;
29 wire status_bar_on;
30 wire status_line_on;
31 wire mode_accent_on;
32 wire alarm_dot_on;
33 wire alarm_flash;

```

```

34
35 wire [7:0] str_char_code;
36 wire [3:0] str_row_idx;
37 wire [7:0] str_row_data;
38 wire      str_on;
39
40 // 字库实例化
41 font_rom u_font_rom (
42     .clk      (clk),
43     .char_code(str_char_code),
44     .row_idx   (str_row_idx),
45     .row_data  (str_row_data)
46 );
47
48 // 字符串渲染参数：放大比例比如倍大小(8，每个字符64像素x128)
49 localparam STR_SCALE = 2;
50 localparam STR_TOP    = 10'd2;
51 localparam STR_LEFT   = 10'd420;
52
53 assign status_bar_on  = (pixel_y < 10'd28);
54 assign status_line_on = 1'b0;
55 assign mode_accent_on = (pixel_x < 10'd8) && status_bar_on;
56 assign alarm_dot_on   = circle_pixel(pixel_x, pixel_y, 10'd604, 10'
d14, 10'd6);
57 assign alarm_flash    = alarm_active && sec[0];
58
59 // 字符选择逻辑
60 // ALARM (A=65, L=76, A=65, R=82, M=77)
61 // CLOCK (C=67, L=76, O=79, C=67, K=75)
62 reg [7:0] char_selected;
63 always @(*) begin
64     if (sw_mode == 1'b1) begin // ALARM
65         case ((pixel_x - STR_LEFT) / (STR_SCALE * 8))
66             0: char_selected = 8'd65; // A
67             1: char_selected = 8'd76; // L
68             2: char_selected = 8'd65; // A
69             3: char_selected = 8'd82; // R
70             4: char_selected = 8'd77; // M
71             default: char_selected = 8'd32; // 空格
72         endcase
73     end else begin // CLOCK
74         case ((pixel_x - STR_LEFT) / (STR_SCALE * 8))
75             0: char_selected = 8'd67; // C
76             1: char_selected = 8'd76; // L
77             2: char_selected = 8'd79; // O
78             3: char_selected = 8'd67; // C
79             4: char_selected = 8'd75; // K
80             default: char_selected = 8'd32; // 空格
81         endcase
82     end
83 end
84
85 // 确定输入ROM
86 assign str_char_code = char_selected;
87 assign str_row_idx = (pixel_y - STR_TOP) / STR_SCALE;
88
89 // 判断当前像素是否在字符串区域内并且对应的像素为1
90 wire [9:0] str_col_idx;

```

```

91     assign str_col_idx = (pixel_x - STR_LEFT) / STR_SCALE;
92
93     assign str_on = (pixel_x >= STR_LEFT) && (pixel_x < STR_LEFT + 5 * 8
94     * STR_SCALE) &&
95     (pixel_y >= STR_TOP) && (pixel_y < STR_TOP + 16 *
96     STR_SCALE) &&
97     str_row_data[str_col_idx % 8];
98
99     assign hour_tens = hour / 10;
100    assign hour_ones = hour % 10;
101    assign min_tens = min / 10;
102    assign min_ones = min % 10;
103    assign sec_tens = sec / 10;
104    assign sec_ones = sec % 10;
105
106    assign colon_on = colon_pixel(pixel_x, pixel_y, 10'd212, 10'd200) ||
107    colon_pixel(pixel_x, pixel_y, 10'd335, 10'd200);
108
109    assign colon_shadow_on = colon_pixel(pixel_x, pixel_y, 10'd216, 10'
110    d204) ||
111    colon_pixel(pixel_x, pixel_y, 10'd339, 10'
112    d204);
113
114    assign text_on = digit_pixel(pixel_x, pixel_y, 10'd112, 10'd200,
115    hour_tens) ||
116    digit_pixel(pixel_x, pixel_y, 10'd160, 10'd200,
117    hour_ones) ||
118    digit_pixel(pixel_x, pixel_y, 10'd235, 10'd200,
119    min_tens) ||
120    digit_pixel(pixel_x, pixel_y, 10'd283, 10'd200,
121    min_ones) ||
122    digit_pixel(pixel_x, pixel_y, 10'd358, 10'd200,
123    sec_tens) ||
124    digit_pixel(pixel_x, pixel_y, 10'd406, 10'd200,
125    sec_ones) ||
126    colon_on;
127
128    assign text_shadow_on = digit_pixel(pixel_x, pixel_y, 10'd116, 10'
129    d204, hour_tens) ||
130    digit_pixel(pixel_x, pixel_y, 10'd164, 10'
131    d204, hour_ones) ||
132    digit_pixel(pixel_x, pixel_y, 10'd239, 10'
133    d204, min_tens) ||
134    digit_pixel(pixel_x, pixel_y, 10'd287, 10'
135    d204, min_ones) ||
136    digit_pixel(pixel_x, pixel_y, 10'd362, 10'
137    d204, sec_tens) ||
138    digit_pixel(pixel_x, pixel_y, 10'd410, 10'
139    d204, sec_ones) ||
140    colon_shadow_on;
141
142    always @(*) begin
143        if (!video_on) begin
144            vga_r = 4'h0;
145            vga_g = 4'h0;
146            vga_b = 4'h0;
147        end else if (str_on) begin
148            vga_r = sw_mode ? 4'hF : 4'h0;
149        end
150    end

```

```

133         vga_g = sw_mode ? 4'h0 : 4'hF;
134         vga_b = 4'h0;
135     end else if (text_on) begin
136         if (alarm_flash) begin
137             vga_r = 4'hF;
138             vga_g = 4'h1;
139             vga_b = 4'h1;
140         end else begin
141             vga_r = 4'hF;
142             vga_g = 4'hF;
143             vga_b = 4'hF;
144         end
145     end else if (text_shadow_on) begin
146         vga_r = 4'h5;
147         vga_g = 4'h6;
148         vga_b = 4'h7;
149     end else if (mode_accent_on) begin
150         if (sw_mode) begin
151             vga_r = 4'hF;
152             vga_g = 4'hB;
153             vga_b = 4'h1;
154         end else begin
155             vga_r = 4'h2;
156             vga_g = 4'hD;
157             vga_b = 4'hC;
158         end
159     end else if (alarm_dot_on) begin
160         if (alarm_flash) begin
161             vga_r = 4'hF;
162             vga_g = 4'h1;
163             vga_b = 4'h1;
164         end else if (alarm_en) begin
165             vga_r = 4'h5;
166             vga_g = 4'hF;
167             vga_b = 4'h7;
168         end else begin
169             vga_r = 4'h5;
170             vga_g = 4'h6;
171             vga_b = 4'h7;
172         end
173     end else if (status_line_on) begin
174         vga_r = 4'hB;
175         vga_g = 4'hC;
176         vga_b = 4'hD;
177     end else if (status_bar_on) begin
178         if (alarm_flash) begin
179             vga_r = 4'hF;
180             vga_g = 4'hD;
181             vga_b = 4'hD;
182         end else begin
183             vga_r = 4'hE;
184             vga_g = 4'hF;
185             vga_b = 4'hF;
186         end
187     end else begin
188         vga_r = bg_r;
189         vga_g = bg_g;
190         vga_b = bg_b;

```

```

191         end
192     end
193
194     function [6:0] digit_segments;
195         input [3:0] digit;
196         begin
197             case (digit)
198                 4'd0: digit_segments = 7'b1111110;
199                 4'd1: digit_segments = 7'b0110000;
200                 4'd2: digit_segments = 7'b1101101;
201                 4'd3: digit_segments = 7'b1111001;
202                 4'd4: digit_segments = 7'b0110011;
203                 4'd5: digit_segments = 7'b1011011;
204                 4'd6: digit_segments = 7'b1011111;
205                 4'd7: digit_segments = 7'b1110000;
206                 4'd8: digit_segments = 7'b1111111;
207                 4'd9: digit_segments = 7'b1111011;
208                 default: digit_segments = 7'b0000000;
209             endcase
210         end
211     endfunction
212
213     function digit_pixel;
214         input [9:0] px;
215         input [9:0] py;
216         input [9:0] left;
217         input [9:0] top;
218         input [3:0] digit;
219         reg [9:0] lx;
220         reg [9:0] ly;
221         reg [6:0] seg;
222         reg a;
223         reg b;
224         reg c;
225         reg d;
226         reg e;
227         reg f;
228         reg g;
229         begin
230             lx = px - left;
231             ly = py - top;
232             seg = digit_segments(digit);
233             a = seg[6] && (lx >= 10'd6) && (lx < 10'd36) && (ly < 10'd6
234 );
235             b = seg[5] && (lx >= 10'd36) && (lx < 10'd42) && (ly >= 10'
236 d6) && (ly < 10'd36);
237             c = seg[4] && (lx >= 10'd36) && (lx < 10'd42) && (ly >= 10'
238 d42) && (ly < 10'd72);
239             d = seg[3] && (lx >= 10'd6) && (lx < 10'd36) && (ly >= 10'
240 d72) && (ly < 10'd78);
241             e = seg[2] && (lx < 10'd6) && (ly >= 10'd42) && (ly < 10'
242 d72);
243             f = seg[1] && (lx < 10'd6) && (ly >= 10'd6) && (ly < 10'
244 d36);
245             g = seg[0] && (lx >= 10'd6) && (lx < 10'd36) && (ly >= 10'
246 d36) && (ly < 10'd42);
247             digit_pixel = (px >= left) && (px < left + 10'd42) &&
248 (py >= top) && (py < top + 10'd78) &&

```



```

242             (a || b || c || d || e || f || g);
243     end
244 endfunction
245
246 function circle_pixel;
247     input [9:0] px;
248     input [9:0] py;
249     input [9:0] cx;
250     input [9:0] cy;
251     input [9:0] radius;
252     reg signed [10:0] dx;
253     reg signed [10:0] dy;
254     reg [21:0] dist2;
255     reg [21:0] radius2;
256     begin
257         dx = $signed({1'b0, px}) - $signed({1'b0, cx});
258         dy = $signed({1'b0, py}) - $signed({1'b0, cy});
259         dist2 = dx * dx + dy * dy;
260         radius2 = radius * radius;
261         circle_pixel = dist2 <= radius2;
262     end
263 endfunction
264
265 function colon_pixel;
266     input [9:0] px;
267     input [9:0] py;
268     input [9:0] left;
269     input [9:0] top;
270     reg [9:0] lx;
271     reg [9:0] ly;
272     begin
273         lx = px - left;
274         ly = py - top;
275         colon_pixel = (px >= left) && (px < left + 10'd12) &&
276                        (py >= top) && (py < top + 10'd78) &&
277                        (((lx >= 10'd2) && (lx < 10'd10) && (ly >= 10'
278 d20) && (ly < 10'd28)) ||
279                        (((lx >= 10'd2) && (lx < 10'd10) && (ly >= 10'
280 d48) && (ly < 10'd56)));
281     end
282 endfunction
283 endmodule

```

Listing 2: anim_gen.v

3 实验结果与分析

完成各功能模块设计后，将Verilog HDL程序综合并下载至实验平台，并通过按键、拨码开关及VGA显示器对系统功能进行测试。测试结果表明，系统能够按照预期完成日常时间计时、时间与闹钟设定、模式状态切换以及VGA视频可视化显示等功能。

在基础计时功能测试中，系统利用高频时钟分频得到的1Hz基准信号进行精确计时。通过VGA显示器可以直观地观察到由于底层计数器的驱动，秒位稳定递增，满60后分钟位准确进位，逻辑状态严密，说明计时核心模块功能完全正确。

在时间调节与设置功能测试中，通过开发板上的独立按键，系统能够分别对“时”和“分”的值进行加减调整。由于内部设计了完善的按键消抖与边沿检测逻辑（脉冲生成），屏幕上时间数字的更新极为精准，每次按压仅执行一次增减操作，无连跳与误触发情况。

在模式切换与闹钟功能测试中，通过拨码开关可以平稳地在正常时钟模式和闹铃设定模式之间切换。处于不同模式时，系统能够正确路由相应的数据，使得屏幕显示的数值即刻切换为对应的时间。当系统当前时间与闹钟设定时间一致，且闹钟使能开关开启时，开发板上的LED指示灯准确点亮，证实了比较与控制逻辑工作正常。

在VGA显示功能测试中，系统成功突破了实体数码管的阻碍，将数字界面映射至屏幕。除了采用模拟七段数码管的方式显示彩色时间数字外，系统还通过引入自建ASCII字库ROM，以不同颜色成功在屏幕上方渲染了“CLOCK”及“ALARM”等模式指示字符串。画面清晰稳定，不仅验证了行场同步时序的精准性，也证明了像素级查表与显像逻辑的高度可靠。

综合测试结果可知，本实验圆满完成了以FPGA为核心的简易VGA数字钟设计。各底层模块间交互顺畅，按键输入控制、时间运算逻辑以及视频图像信号输出的整个流程完整无误。实验现象与原设计预期完全一致，深刻验证了Verilog HDL在时序控制以及复杂数据视频输出上的强大应用能力。

4 测试结果照片

5 遇到的问题与解决办法

5.1 VGA字库显示水平翻转问题

在向简易数字钟的VGA显示模块中引入字库（Font ROM）以增加模式指示字幕（ALARM和CLOCK）时，最初发现屏幕上显示的字母发生了水平镜像翻转。这是由于字库生成脚本中提取字模每一行像素的顺序（从左到右或从右到左）与VGA扫描控制逻辑中像素索引的提取顺序不一致所导致的。

解决办法：为了快速在不重新生成字模文件的情况下解决该显示问题，我在Verilog代码 `anim_gen.v` 中的像点拾取逻辑中加入了反转映射。具体而言，将原来直接使用的列索引进行位翻转：使用 `3'd7 - str_col_idx[2:0]` 替代原来的直接索引。这样调整后，使得VGA驱动在进行从左向右的电子束扫描时，能以正确的顺序逐位读出字形数据，从而纠正了字幕翻转的现象。

5.2 VGA显示系统的前后景叠加与优先级处理

数字钟的可视化部分除了单纯的文字外，还涉及到动态生成的数字、辅助用的标点以及作为底色的背景。初期由于各种图形元素的绘制定界独立并行存在，且在最后赋值RGB信号时采用了简单的条件覆盖，当时间数字恰好覆盖于复杂背景及模式字模区域时，发生了图形交替闪烁，甚至显示撕裂的情况，前后景像素优先级出现了混乱。

解决办法：梳理并在代码层次明确各个图形叠加的优先级，使用分支结构在每个像素周期的组合逻辑中构建层级判断。具体而言，针对给定的 `pixel_x` 和 `pixel_y`，首先优先判断且渲染最上层的前景色，如果不命中该区域再依次判断是否命中时间数字或间隔冒号，若前面的前景条件均不满足，最末才根据坐标从 `blk_mem_gen` 例化的显存读取背景像素值输出至显示器。这样确保了每一次电子束扫描都能获得唯一的、层级最高的RGB输出，彻底解决了画面穿模与撕裂的现象。